

BACKWARD CHAINING

Aim:

To implement the Backward Chaining algorithm in Prolog.

Objective:

To prove a given goal by recursively checking the conditions required to establish it.

Algorithm:

- 1. Start.**
- 2. Specify the goal.**
- 3. Check whether the goal is an existing fact.**
- 4. If yes, the goal is proved.**
- 5. Otherwise, find a rule whose conclusion matches the goal.**
- 6. Treat the rule's conditions as subgoals.**
- 7. Recursively prove all subgoals.**
- 8. If all subgoals are proved, the goal is proved.**
- 9. Otherwise, backtrack and try another rule.**
- 10. Stop.**

Flowchart:

START

|

v

Set Goal

|

v

Is Goal a Fact?

/ \

Yes No

| |

Proved Find Rule

| |

| **Rule Found?**

| / \

| **No Yes**

| | |

| **Fail Create Subgoals**

| |

| **Prove Subgoals**

| |

| **All True?**

| / \

| **Yes No**

| | |

| **Proved** **Backtrack**

\ /

\ /

STOP

Prolog Program:

% Backward Chaining

fact(fever).

fact(cough).

rule(flu, [fever, cough]).

rule(doctor_visit, [flu]).

rule(treatment, [doctor_visit]).

backward(Goal) :-

prove(Goal, []).

prove(Goal, _) :-

fact(Goal),

write('Fact found: '),

write(Goal),

nl.

prove(Goal, Visited) :-

\+ fact(Goal),

\+ member(Goal, Visited),

rule(Goal, Conditions),

```
write('Trying to prove: '),
```

```
write(Goal),
```

```
nl,
```

```
prove_all(Conditions, [Goal|Visited]).
```

```
prove_all([], _).
```

```
prove_all([H|T], Visited) :-
```

```
    prove(H, Visited),
```

```
    prove_all(T, Visited).
```

Query

```
?- backward(treatment).
```

Output

Trying to prove: treatment

Trying to prove: doctor_visit

Trying to prove: flu

Fact found: fever

Fact found: cough

true.

Another query:

```
?- backward(pneumonia).
```

Output:

```
[1] 99 ?- prove(pneumonia).  
Goal: pneumonia  
Required conditions: [cough,breathlessness]  
Fact found: cough  
Fact found: breathlessness  
[1] 99 ?- prove(pneumonia).
```

Result:

Thus, the Backward Chaining algorithm is successfully implemented to prove a goal by recursively satisfying its required conditions.